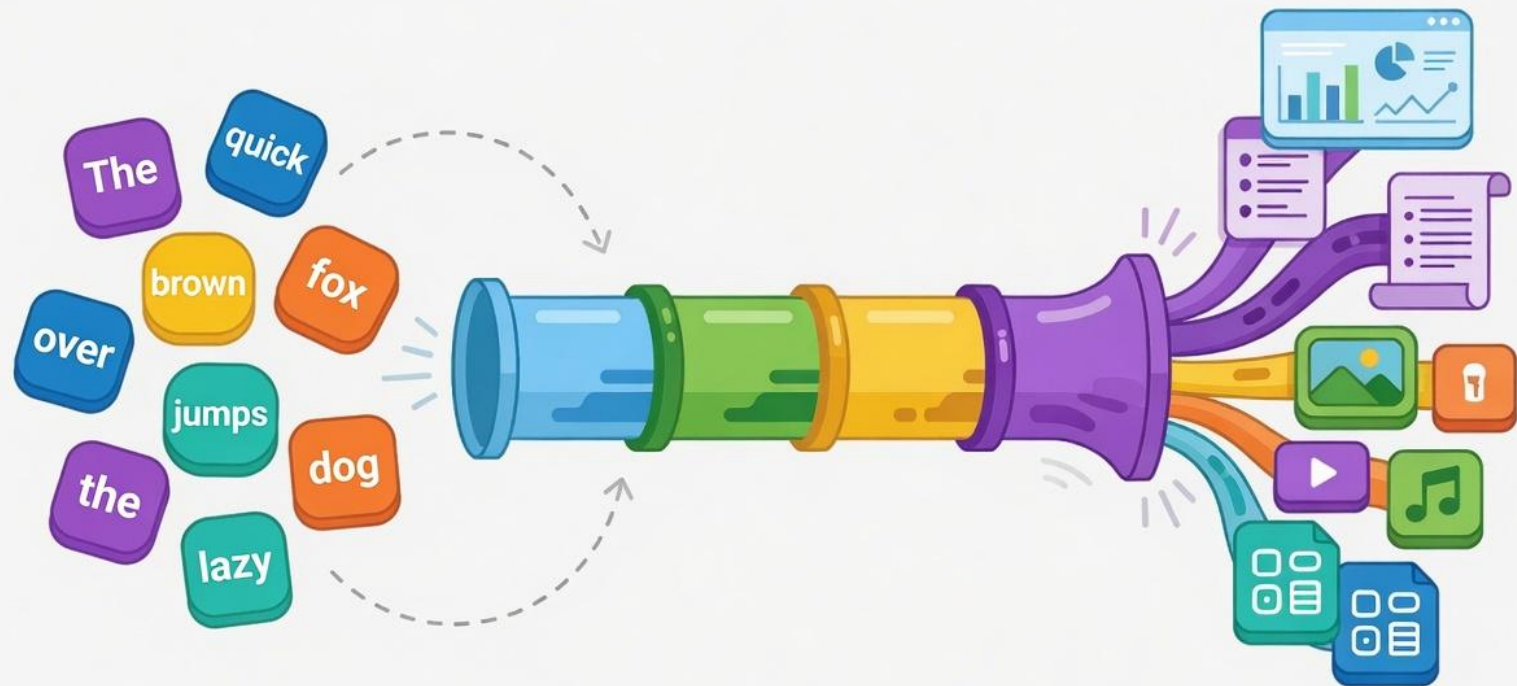


Token Efficiency





TOKEN
LEGEND

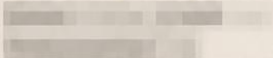


INVOICE

DATE: AUGUST 2024
INVOICE # CLAUDE-08-2024

BILL TO:

VALUED CUSTOMER



DESCRIPTION

AMOUNT (USD)

Claude API Usage

(July 1 – July 31, 2024)

\$500,000,000.00

Usage-based charges for API access,
models, compute, and related services.

TOTAL DUE	
\$500,000,000.00	
Thank you for building with Claude. We appreciate your partnership.	
SUBTOTAL	\$500,000,000.00
TAX (0%)	\$0.00
TOTAL DUE (USD)	\$500,000,000.00



Anthropic PBC
548 Market St, PMB 97275
San Francisco, CA 94104
United States
billing@anthropic.com

PAYMENT TERMS

Due upon receipt.

ANNOUNCEMENT

Linux Foundation Announces the Intent to Launch the Tokenomics Foundation to Establish Open Standards for AI Cost Management





= 4 tokens

1. Cost is invisible during design

You only see it on the bill later.

```
ChatCompletion.create(  
  model: "gpt-4o",  
  messages: [  
    { role: "system", content: ... },  
    { role: "user", content: ... }  
  ],  
  max_tokens: 1000  
  temperature: 0.2  
)
```

?

What will
this cost?

 Your Invoice

April 1 - April 30

Total

\$1,842.72



2. Three token types. Three wildly different prices.

Every call is a mix — and the rates change constantly.

	Input Tokens (you pay)	Cached Input Tokens (you pay less)	Output Tokens (you pay more)
This call	8,542	45,128	1,203
Price / 1K tokens (example)	\$0.015	\$0.0015	\$0.060
Cost for this call	\$0.128	\$0.068	\$0.072

Rates change (live)



3. Rates vary by model, region, and tier.

There's no single price to memorize.

Live pricing (per 1K tokens)

Model

All models

Region

All regions

Tier

All tiers

Model	Region	Tier	Input (You pay)	Cached Input (You pay less)	Output (You pay more)	Last updated
gpt-4o	East US	Standard	\$0.015	\$0.0015	\$0.060	2 min ago
gpt-4o	East US	Provisioned	\$0.010	\$0.0010	\$0.040	2 min ago
gpt-4o	Sweden Central	Standard	\$0.016	\$0.0016	\$0.064	1 min ago
gpt-4.1	East US	Standard	\$0.003	\$0.0003	\$0.012	1 min ago
gpt-4.1-mini	East US	Standard	\$0.0008	\$0.0001	\$0.0032	1 min ago

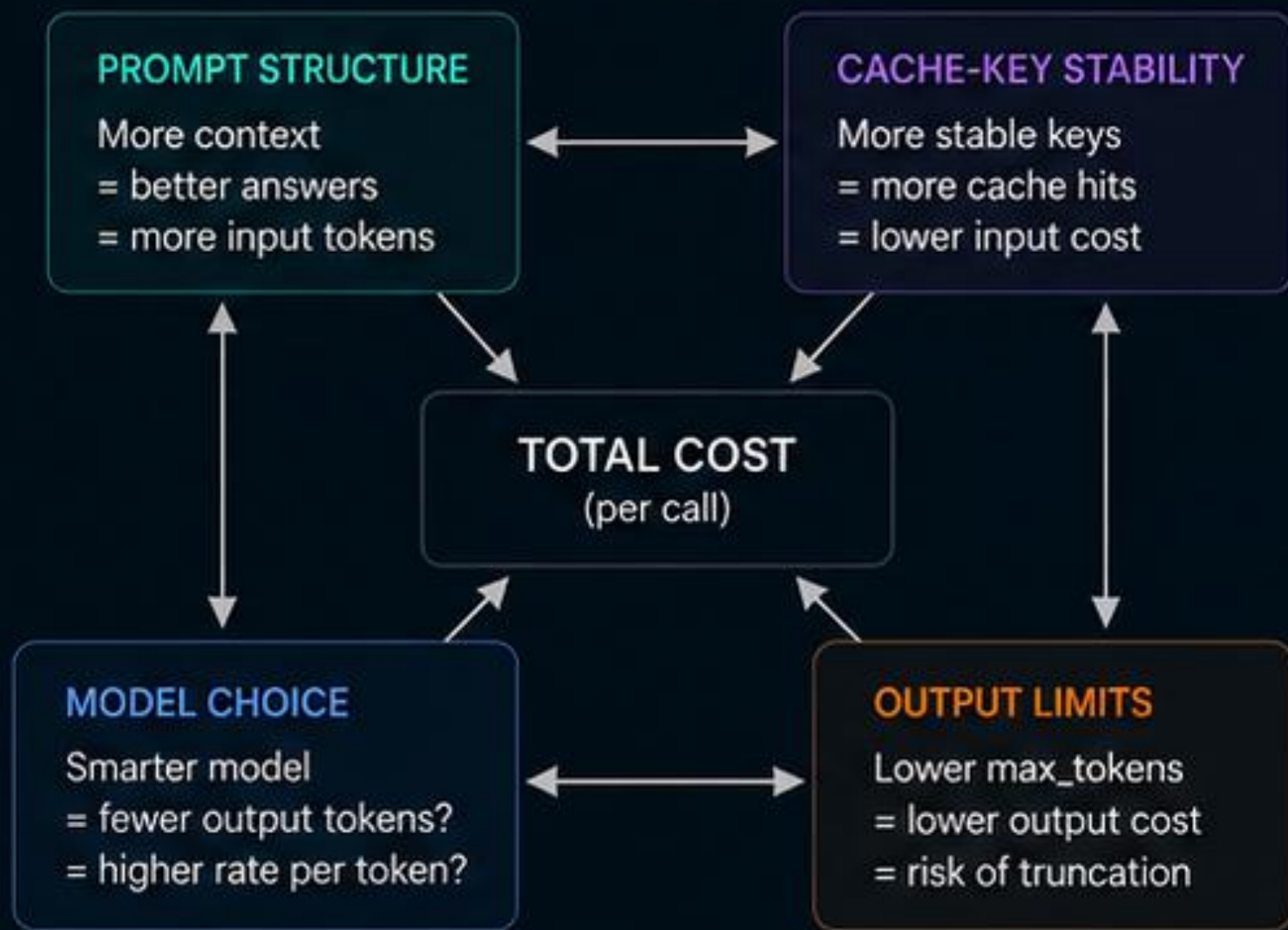
4. You're juggling multiple interacting levers.

Change one thing, and everything shifts.



5. The levers interact in non-obvious ways.

Optimize one, and another might get worse.



6. You can't tune what you can't see.

You need per-call visibility to get it right.

Per-call token breakdown

Total cost

\$0.2687

Input tokens

12,351

\$0.1853

Cached input tokens

37,842

\$0.0568

Output tokens

1,102

\$0.0666

Breakdown

Type	Tokens	Rate / 1K	Cost	% of total
Input	12,351	\$0.0150	\$0.1853	69%
Cached input	37,842	\$0.0015	\$0.0568	21%
Output	1,102	\$0.0605	\$0.0666	25%
Total	51,295	—	\$0.2687	100%

What-if simulation

Max output tokens



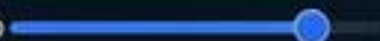
1000

Reduce input tokens by



-20%

Increase cache hit rate to



80%

Estimated new cost

\$0.1762

↓ 34%



← ↻  https://prices.azure.com/api/retail/prices



Work items - Boards



(4) Discord | Friends



N

Pretty-print ☒

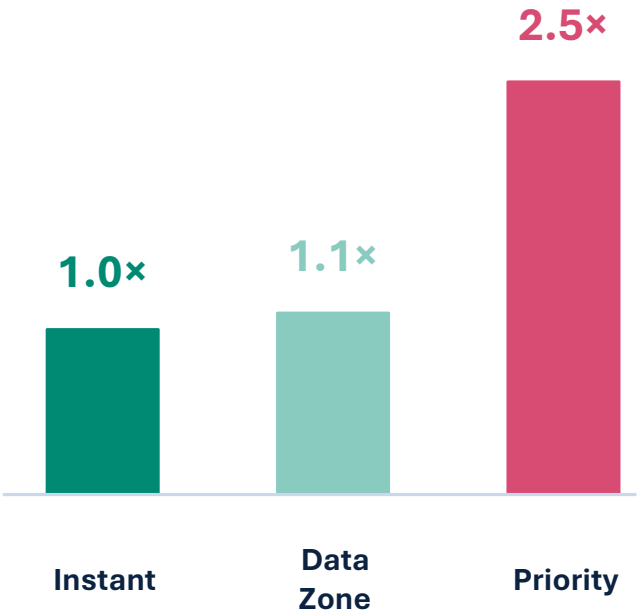
```
  },
  {
    "currencyCode": "USD",
    "tierMinimumUnits": 0,
    "retailPrice": 2.5,
    "unitPrice": 2.5,
    "armRegionName": "polandcentral",
    "location": "PL Central",
    "effectiveStartDate": "2026-05-01T00:00:00Z",
    "meterId": "000efa89-8bd5-540e-9693-12d6d9d6a50",
    "meterName": "5.5 ShortCo Batch inp G1 1M Token",
    "productId": "DZH318Z0T9WD",
    "skuId": "DZH318Z0T9WD/0NJ5",
    "productName": "Azure OpenAI GPT5",
    "skuName": "5.5 ShortCo Batch inp G1",
    "serviceName": "Foundry Models",
    "serviceId": "DZH31600VF1F",
    "serviceFamily": "AI + Machine Learning",
    "unitOfMeasure": "1M",
    "type": "Consumption",
    "isPrimaryMeterRegion": true,
    "armSkuName": "55 ShortCo Batch inp G1"
  },
  {
```

INSTANT VS DEPLOYED PRICING

OPTION	INPUT	OUTPUT	RATE
Instant model NO DEPLOYMENT REQUIRED	USD 5.00 IN	USD 30.00 OUT	USD 0.001235 BASELINE
Data Zone deployment PINNED TO DATA-ZONE DEPLOYMENT	USD 5.50 IN	USD 33.00 OUT	USD 0.001358 1.1× COST
Priority processing PREMIUM PRIORITY-PROCESSING TIER	USD 12.50 IN	USD 75.00 OUT	USD 0.003087 2.5× COST

Relative cost

Blended example rate



Prompt Caching

Cache init, then reuse

Turn one big, repeated prompt into cheap cached input tokens.



standard input

cached input (cheaper)

THE IDEA

One big stable prompt, reused every call

STABLE PREFIX

reference material · ~120 identical sections · same every call

TAIL

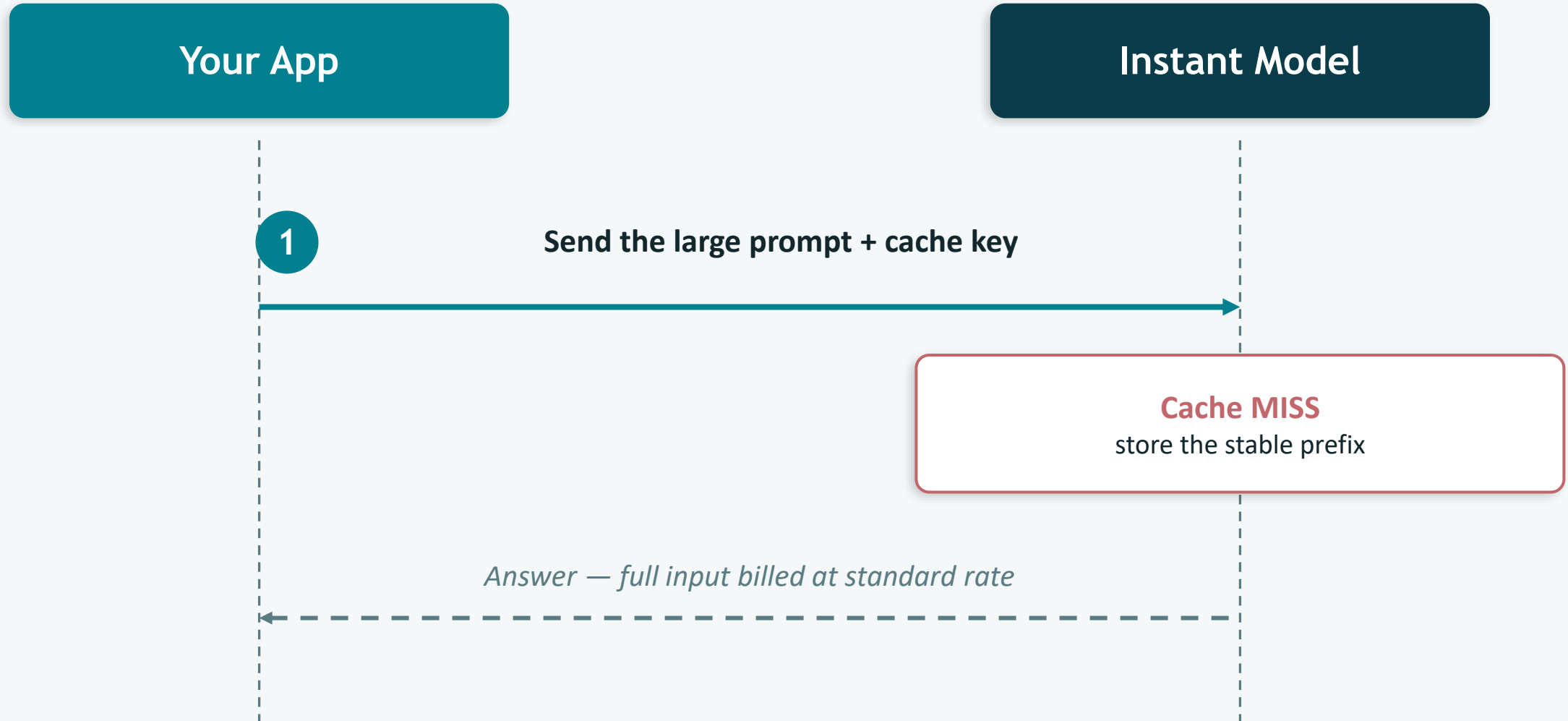
run id + question

cache key **im-cache-`<runId>`**

Same prefix on every call → the model can cache it and skip re-charging full price for those tokens.

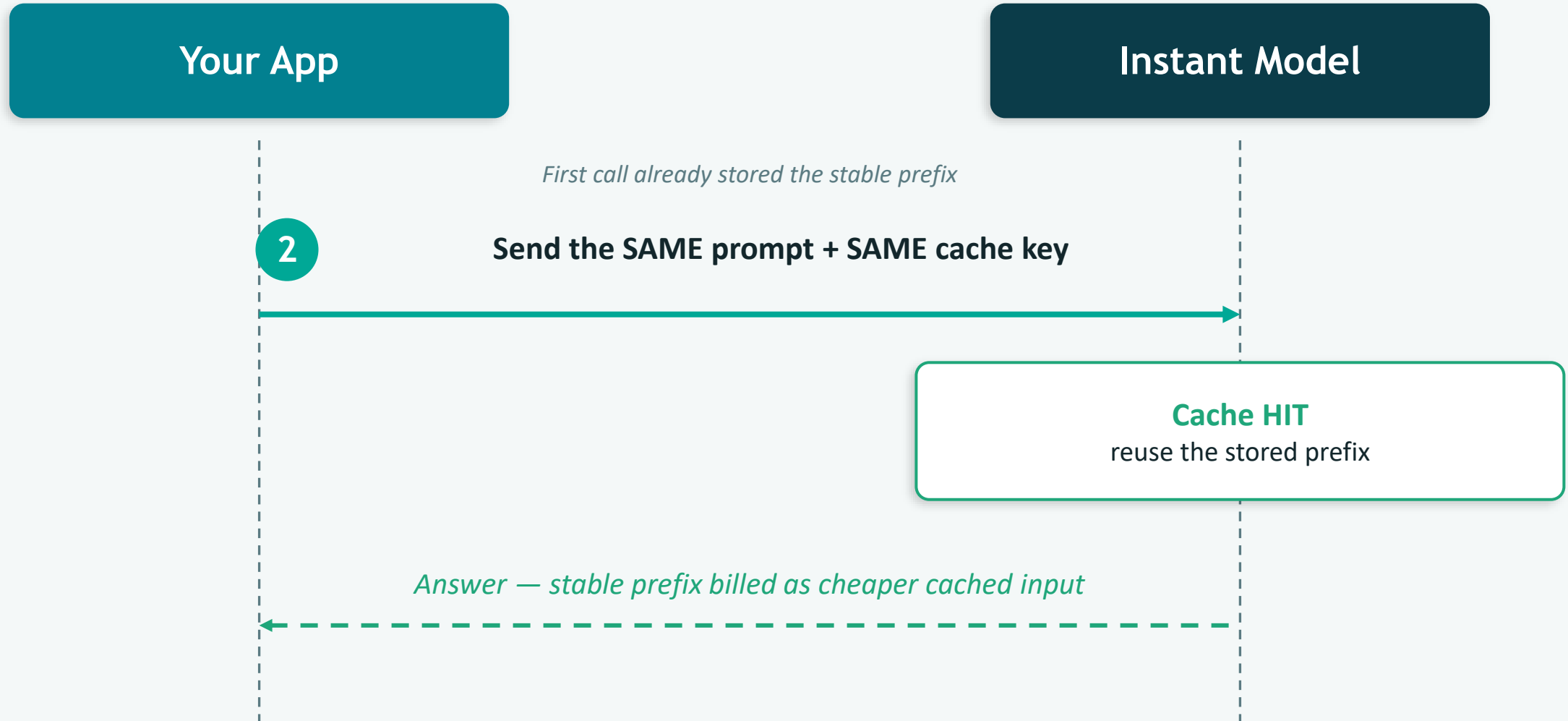
STEP 1

Cache init — the first call



STEP 2

Reuse – the second call



WHY IT MATTERS

Reuse the prefix, pay less

1 Stable prefix

The big repeated part is billed as cached input on later calls — cheaper than standard input.

2 Only the tail changes

A tiny unique tail (run id + question) is the only standard-rate input each call.

3 Repeated calls win

More reuse of the same prefix means lower cost and faster responses.

Compaction

Shrink a big context into a small, durable one

Rewrite long working notes into a compact summary that carries the same meaning — then reuse it every turn.



Compaction

Compact the submitted working notes into a concise durable context summary for a future assistant turn. Use at most six compact sentences, no markdown bullets, and no nested lists. Preserve only next-turn essentials: goal, key facts, docs or files to update, validation and deploy commands, blockers, and privacy constraints. Remove repetition, resolved dead ends, greetings, transient logs, and exact values that are not needed for the next step. Summarize quota findings as sanitized quota evidence instead of repeating every number. Omit optional follow-ups unless they are required for the next action. Return only the compacted summary.

Caveman

Answer like a caveman to save output tokens, inspired by the caveman skill: why use many token when few do trick. Drop filler words, articles, and pleasantries; use short telegraphic fragments instead of full sentences. Keep every technical fact, step, and caveat accurate. Brain still big, mouth small. Keep code, commands, file paths, API names, and error strings exact and unchanged. Reply in the same language as the question. Compress the style, not the meaning.

WHAT IT DOES

One call: long context in, small context out

BEFORE — long working notes



795 tokens



-62%

AFTER — durable summary

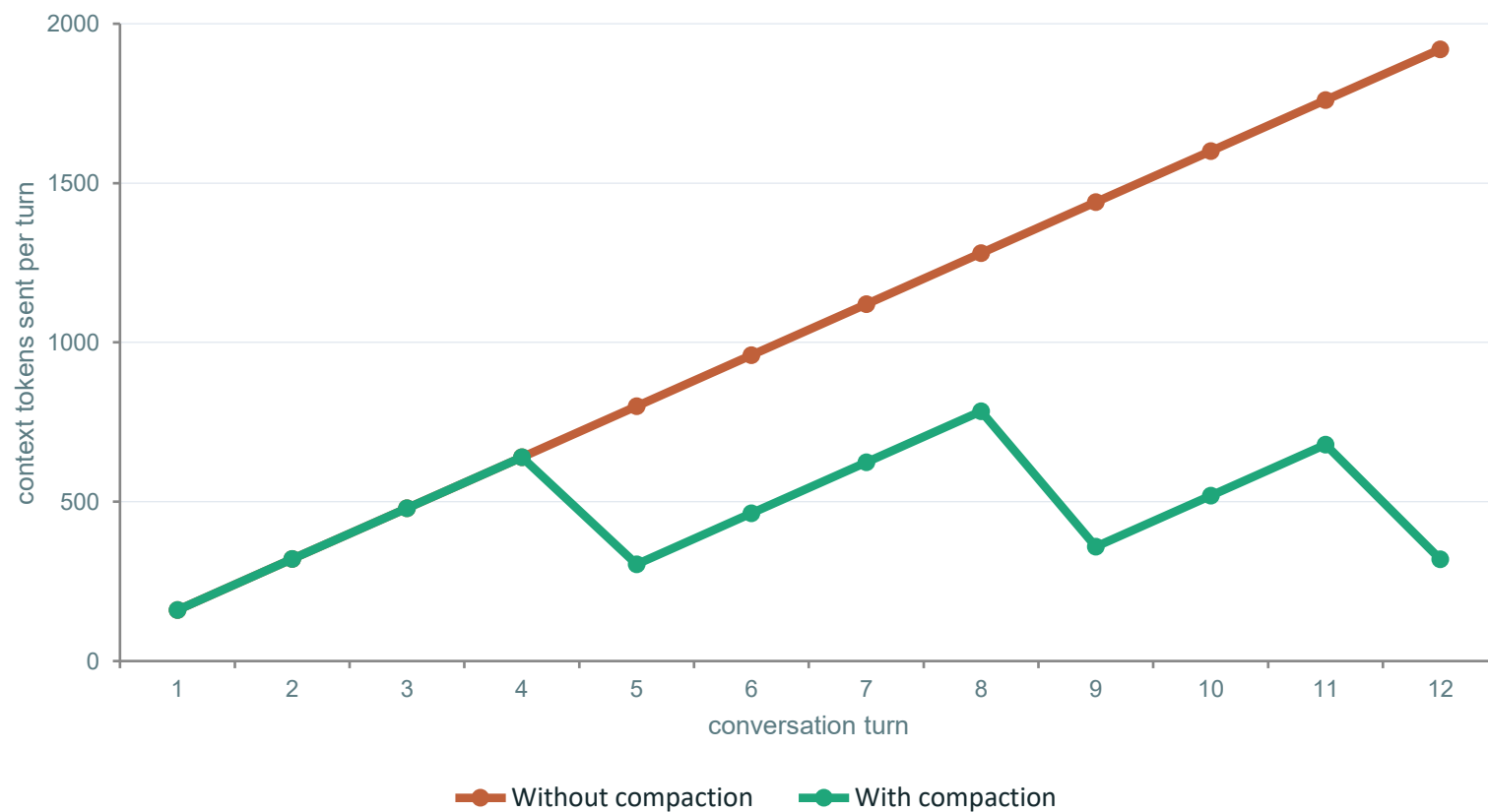


302 tokens

WHY IT MATTERS

Context stays small instead of growing every turn

Illustrative projection · the -62% summary size is measured live (795 → 302 tok)



Without

history re-sent whole every turn → 1,920 tok by turn 12

With

summary resets the context near 795 tok → never climbs

TAKEAWAY

Compact the context, keep the meaning

1 What

Rewrites long notes into a short durable summary.

2 When

Long agent conversation or working log keeps growing and you carry it across turns.

3 Payoff

Fewer input tokens on every future turn, and context stays bounded instead of climbing.



AI TOKEN COSTS

- A PIRATE'S GUIDE TO WHAT LIES AHEAD -




INPUT TOKENS
The words ye send
- Cheap at first -


CONTEXT WINDOW
The more ye carry,
the heavier the toll.
\$\$\$


PROMPT ENGINEERING
Crafty words save tokens,
sloppy words sink ships.


COMPLETION TOKENS
The AI's reply.
The bigger the answer,
the deeper the cost.


CACHE & OPTIMIZATION
Smart sailors cache
and reuse.
Save yer gold!



SECRET PHRASE
"/chronicle
/limits,"

LEGEND

- THE COSTLY PATH
-  TOKEN FEES
-  HIDDEN DANGERS
-  CONTEXT CURRENTS

"He who knows the tokens, "knows thes
He who knows the limits, thiélts"(cornere jafs.

THE SECRET PHRASE
"/chronicle
/limits
Speak these words
at the right time,
and all shall be revealed.





Developer

/chronicle

WORKS ACROSS

 Copilot CLI

 Copilot App

 VS Code

 JetBrains IDEs

 Copilot SDK



COPILOT /CHRONICLE

Your work history. Actionable insights.



Standup

Summaries of your recent work



Search

Find topics, decisions, or changes



Tips

Personalized advice to work better



Cost Tips

Find patterns and reduce AI spend



Improve

Improve your copilot-instructions.md



Reindex

Rebuild and sync your history



SESSION STORE

Private. Secure. Yours.



Prompts & responses



Tools & commands



Files & changes



Token usage



Costs & metadata



Know what changed

See what you worked on and why



Find anything

Search across your past sessions



Work smarter

Get tips based on your patterns



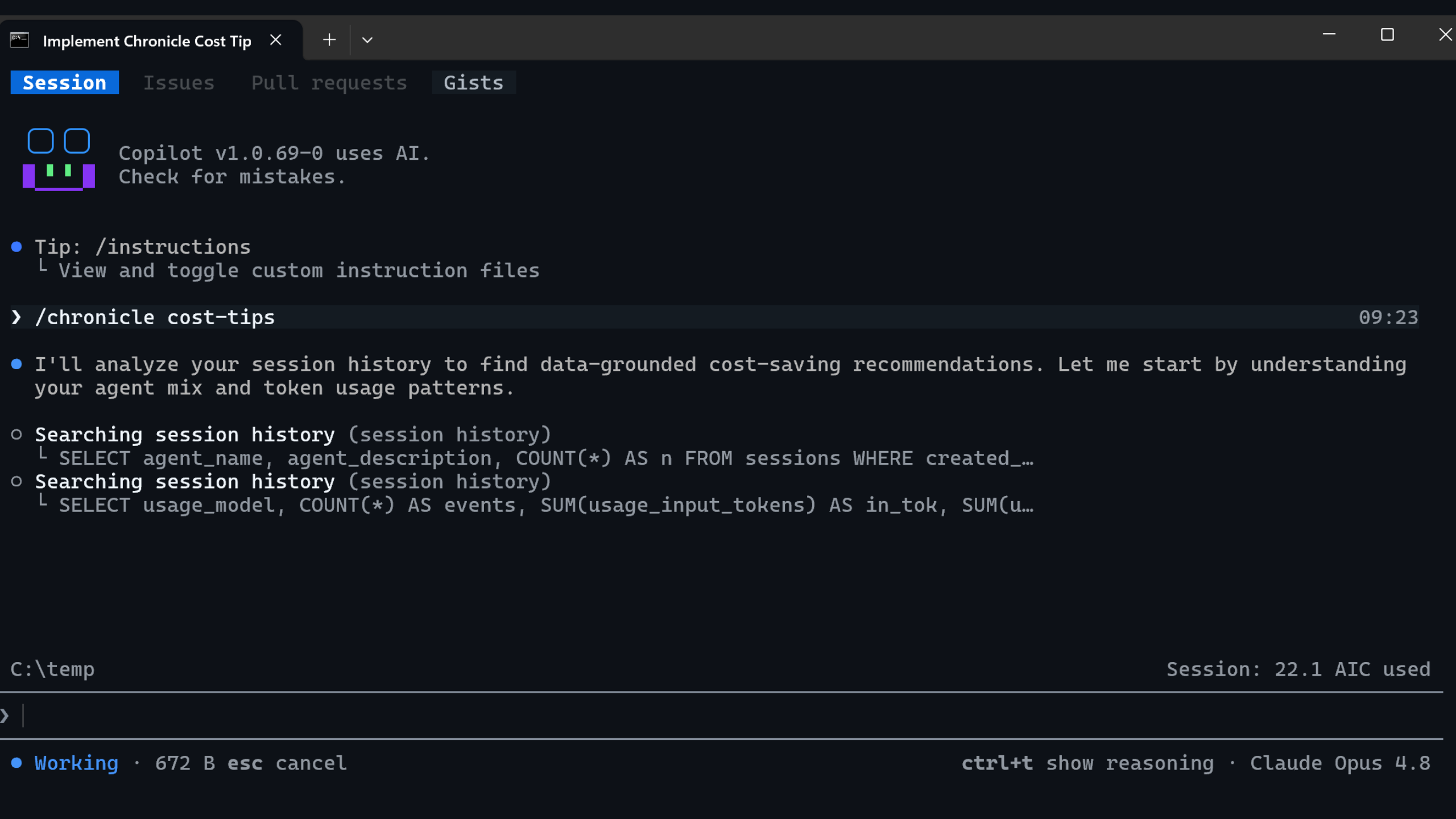
Spend less

Understand and reduce AI costs



Make it stick

Turn patterns into better instructions



Copilot v1.0.69-0 uses AI.
Check for mistakes.

- Tip: /instructions
 - ↳ View and toggle custom instruction files

> /chronicle cost-tips

09:23

- I'll analyze your session history to find data-grounded cost-saving recommendations. Let me start by understanding your agent mix and token usage patterns.
 - Searching session history (session history)
 - ↳ SELECT agent_name, agent_description, COUNT(*) AS n FROM sessions WHERE created_...
 - Searching session history (session history)
 - ↳ SELECT usage_model, COUNT(*) AS events, SUM(usage_input_tokens) AS in_tok, SUM(u...

C:\temp

Session: 22.1 AIC used

- Analysis complete. Here's what your data shows and how to cut spend. You only drive one interactive agent – Copilot CLI (157 sessions; your 20 Code Review sessions are automated and excluded). Over 60 days, input tokens dwarf output ~100:1, the classic "paying to re-send a bloated context every turn" signature.

Where your tokens actually go

- Opus 4.8: 45.8M input tokens (top premium spend), avg 87K input/event
- Haiku 4.5: 41.7M input (you already route a lot here – good)
- A handful of long sessions dominate: one hit 14.4M input tokens alone.

Tip 1 – You're not compacting long sessions. This is your biggest leak.

Session `a3a0e387` (astro-coffee issue #4) climbed monotonically from 49K → 174K input tokens over 66 turns with zero `/compact` – auto-compaction only fired at turn 67, near the end. Every turn after ~30 was re-sending 120–174K tokens on Opus 4.8.

Fix: Run `/compact` around turn 25–30 when `/context` shows ~110K. Dropping back to ~50K would have shaved ~90–120K tokens off *each* of the remaining ~35 turns – roughly 3M input tokens saved in that one session. Watch `/context`; compact whenever it crosses ~50% before your next big turn.

Tip 2 – Four sessions re-explored the *same* issue from scratch.



Developer

/limits



COPILOT /LIMITS

Control AI usage. Avoid surprises.



1. Set or update limit

Choose budget per session (USD)



2. Track usage

Monitor tokens and cost in real time



3. Enforce limit

Warns at threshold,
stops at limit



4. You stay in control

Modify, raise, or remove
limits anytime



LIMITS BUDGET STORE

Private. Secure. Yours.



Per user & per session



Budget amount (USD)



Current spend



Alert thresholds



Resets with new session



Encrypted & private

WORKS ACROSS



Copilot CLI



VS Code



JetBrains IDEs



Copilot App



Copilot SDK



Prevent bill shock

Never exceed
your budget



Full visibility

See spend, remaining,
and session details



Full control

View, set, update,
or remove limits



Team friendly

Helps individuals
manage costs



Optimize spend

Use the right model
for the right work

**Session**

Issues

Pull requests

Gists

What remains (next session):

- You said "yes" to a fix, but I can't run tools now. The lowest-risk change to start with is adding a connect/request timeout to `RetailPricingClient`'s `HttpClient` / `HttpRequest`. After that, externalizing the meter-match tokens (#2) is the highest-value robustness fix.
- Unreviewed areas: `DemoRunService` (21.9 KB), `DemoController`, `TokenTelemetry`, the four UI flows, and the Bicep infra.

Kick off a new session and say "add the HTTP timeout" and I'll implement + `mvn test` it.

- Session limit reached. Increase or unset the session limit to continue.

> try again

09:45

- Session limit reached. Increase or unset the session limit to continue.

Session limit reached

This session has used 34.7 of 30 AI credits. Increase or remove the session limit to continue.

- > 1. Add AI credits**
- 2. Set new AI credit limit
- 3. Remove session limit
- 4. Cancel (Esc)

↑/↓ to navigate · enter to select · esc to cancel



Project Setup

- devcontainer.json
- Copilot instructions
- **A**gent profile



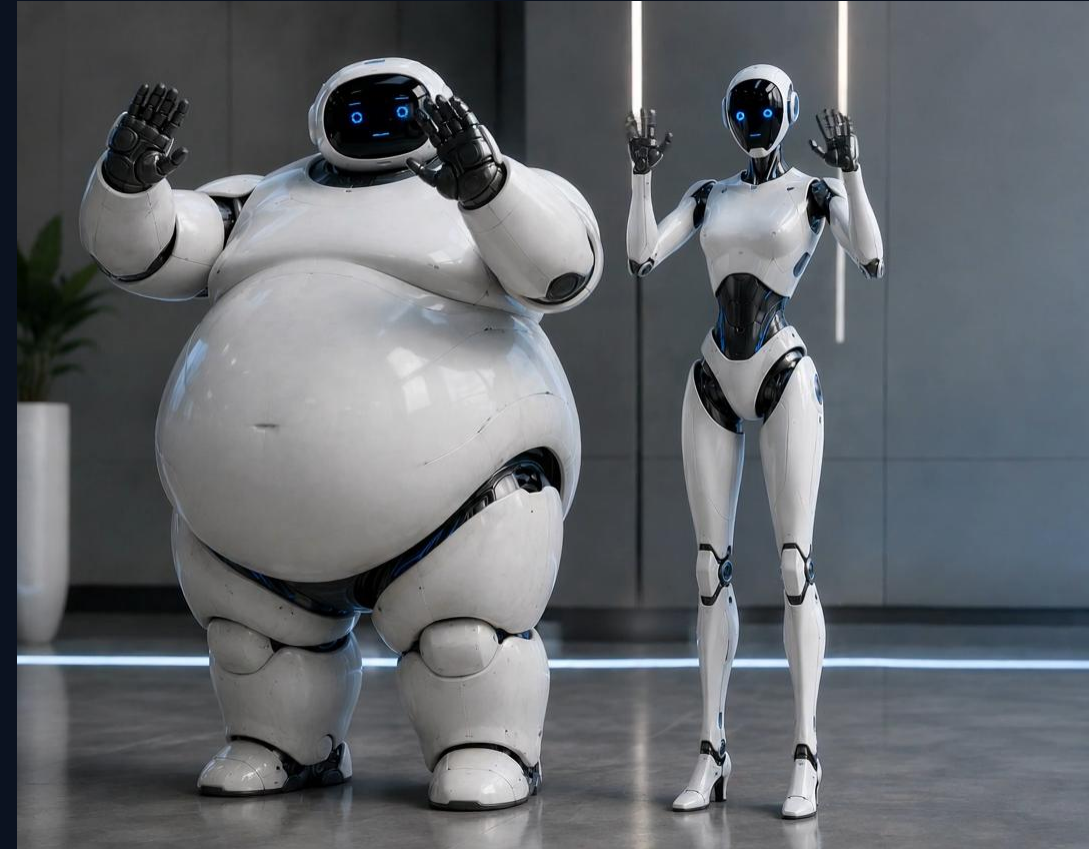
Reduce input and context tokens

- Drop unused MCP tools
- Scope sessions tightly
- Keep prompts precise and short



Choose the right execution path

- Use the smallest model that fits
- Prefer a direct completion
- Watch output, not just input





Token Efficiency

Run an instant model call, compare prompt cache reuse, then compact long working notes into a smaller reusable prompt.

MODEL
gpt-chat-latest

METER
5.5 ShortCo

REGION
westus3 / GL

LIVE PRICING
Instant Demo

An **instant model** is called by name — there's no deployment to create or manage, and it's available to every Foundry project automatically. It bills at standard pay-as-you-go rates, so it stays much cheaper than a data-zone or priority deployment. This call is priced live against Azure Retail Prices meters, then scaled to 1,000 and 1M runs.

1 call

Prompt

In one sentence, why are instant models useful for prototyping?

REUSABLE PREFIX
Prompt Cache Demo

Watch the model cache warm up in real time. A warm-up call primes a long stable prefix, then the repeated call reuses it — see the cache fill, the cold→warm comparison, and exactly what was loaded.

2 calls

Warm the cache

CONVERSATION HYGIENE
Compaction Demo

Long working notes pile up tokens on every turn. One call compacts them into a short, durable summary that keeps only the next-turn essentials — goal, key facts, and commands — so future prompts stay cheap. See the before→after token drop and reduction rate.

1 call

Working notes

Goal: ship a small Java dashboard change that explains instant model throughput limits.
Context gathered: Microsoft Foundry instant models are preview-scoped to West US 3 and can be called by model

Demo

<https://aka.ms/costs>

